

Atomic AI Security Units: Configuration-Bound Security Evaluation and Governance for Tool-Using and Retrieval-Augmented LLM Systems

Author Name(s)

Organization

email@domain.com

February 23, 2026

Abstract

Enterprise generative AI systems increasingly combine prompt packages, large language models (LLMs), retrieval-augmented generation (RAG), and tool execution (plugins, connectors, or agent toolchains). Security failures in these systems are rarely attributable to the base model in isolation; instead they arise from configuration choices and from emergent behavior across multi-component orchestration topologies. We introduce the *Atomic AI Security Unit* (AASU), a formal abstraction that treats a specific, versioned configuration snapshot—prompt package, model parameters, retrieval configuration, tool interfaces, and runtime constraints—as the minimal unit of security evaluation, certification, and governance. We model deployed AI applications as directed graphs of AASUs, show how threat modeling and testing must be bound to both node configurations and edge-level data flows, and propose a three-layer validation methodology (unit, orchestration, and attack-graph) with auditable change control via configuration manifests and hashes. We situate AASU within prior work on prompt injection, indirect prompt injection benchmarks, tool-integrated agent attacks, and emerging testing frameworks, and we map AASU components to widely used taxonomies and governance frameworks.

Introduction

LLM-enabled applications have moved from single-model chat interfaces to composite systems that integrate: (i) a prompt *package* (including system/developer prompts, routing prompts, and policies), (ii) a model instance and decoding/runtime parameters, (iii) retrieval layers (vector databases, corpora

selection, filters, and thresholds), and (iv) tool execution layers that can read and write to enterprise systems. This composition introduces new security failure modes such as direct and indirect prompt injection, malicious instruction propagation via retrieved content, and privilege escalation through tool chaining and agent routing.

Existing assurance practices often treat the model as the unit under test, while operational systems change prompts, tool schemas, retrieval indices, and guardrails on a much faster cadence than model upgrades. Recent work has shown that prompt injection attacks are practical against LLM-integrated applications (Liu et al. 2023) and that indirect prompt injection is particularly challenging in real-world data-flow settings (Hartmann and Gurevych 2023). Meanwhile, tool-using agents (e.g., ReAct-style systems (Yao et al. 2022) and tool-learning approaches (Schick et al. 2023)) expand the attack surface from text generation to action execution, shifting security questions from output quality to capability safety and least privilege.

Contributions.

This paper makes four contributions:

1. We define the AASU as a configuration-bound, versioned unit of security evaluation: $AASU = (P, M, R, T, K, H, S)$.
2. We model deployed LLM applications as directed graphs of AASUs and formalize node- and edge-scoped threat surfaces.
3. We propose a three-layer validation methodology and coverage metrics suitable for enterprise assurance.
4. We describe governance controls to make test results auditable and change-aware via manifests and configuration hashing.

Background and Related Work

Tool-Using and Retrieval-Augmented LLM Systems

Tool-using paradigms such as ReAct (Yao et al. 2022) and self-taught tool use (Toolformer) (Schick et al. 2023) enable LLMs to call external APIs, search systems, and enterprise services. Retrieval-augmented generation (RAG) (Lewis et al. 2020) injects retrieved context into model prompts to improve factuality and domain grounding, but also introduces a new input channel for adversarial content and data exfiltration risks.

As agent frameworks evolve, standardized tool interoperability layers (e.g., Model Context Protocol) reduce integration friction, but also make tool supply chains and permission surfaces more uniform and therefore more scalable for both defenders and attackers (Model Context Protocol 2024; Anthropic 2024).

Prompt Injection and Indirect Prompt Injection

Prompt injection attacks exploit the fact that LLMs treat instructions in context as actionable, even when untrusted content is mixed with system guidance. Liu *et al.* show practical prompt injection attacks against LLM-integrated applications and discuss mitigations (Liu et al. 2023). Hartmann and Gurevych introduce BIPIA, a benchmark for *indirect* prompt injection, where malicious instructions arrive through external content such as webpages or documents (Hartmann and Gurevych 2023). Recent benchmarks for tool-integrated agents, including InjecAgent, explicitly focus on indirect prompt injection and tool misuse behaviors (Liao et al. 2024).

Defensive approaches include structured querying and instruction/context separation (e.g., StruQ) (Arocas, Lembke, and Heintz 2024) and alignment strategies designed to be robust against injection (SecAlign) (Wu et al. 2024). Design-time approaches that constrain agent capabilities and instruction channels (e.g., CaMeL) aim to make injection failures less catastrophic even when attacks succeed (Tramèr et al. 2025). However, empirical results across defenses are highly configuration-dependent and can fail under distribution shifts in tools, corpora, or routing logic.

Security Testing Frameworks and Red Teaming

Security probing and red teaming for LLM systems is increasingly automated. Examples include garak (Derczynski et al. 2024), PyRIT (Kumar et al. 2024), and CyberSecEval (Jiang et al. 2023), which provide test harnesses and datasets for adversarial evaluation. These systems often assume a target endpoint, but enterprise deployments frequently have many configuration variants of an application (prompt versions, tool policies, retrieval indices), making it difficult to bind outcomes to the correct unit of change without an explicit configuration abstraction.

Governance and Compliance Context

Governance frameworks increasingly emphasize risk management, documentation, and change control for AI systems. Notable examples include NIST AI RMF (National Institute of Standards and Technology 2023), the NIST Generative AI Profile (National Institute of Standards and Technology 2024), and emerging management system standards such as ISO/IEC 42001 (International Organization for Standardization 2023). Security taxonomies such as the OWASP Top 10 for LLM Applications (OWASP Foundation 2023) and MITRE ATLAS (MITRE 2020) provide shared vocabulary for risks and techniques but do not directly prescribe how to bind tests and approvals to fast-changing configurations. Regulatory regimes (e.g., the EU AI Act) further increase the need for traceable evidence linked to deployed system versions (European Union 2024).

Atomic AI Security Units

Definition

We define an AASU as the minimal configuration snapshot that must be treated as a single unit for security evaluation:

$$\text{AASU} = (P, M, R, T, K, H, S)$$

where:

- P is the prompt package (system/developer prompts, policy prompts, templates, routing prompts, and prompt hierarchy),
- M is the model instance and parameters (provider, model version, decoding settings),
- R is the retrieval configuration (corpora, embedding model, chunking, filters, top- k , thresholds),
- T is the tool interface and execution configuration (schemas, permissions, allow/deny lists, connector versions, runtime sandbox),
- K is runtime constraints and guardrails (rate limits, timeouts, context limits, output handling constraints),
- H is the history/memory configuration (state backend, retention, and isolation controls),
- S is the skill configuration (skill pack definitions, invocation policy, and skill-scoped privileges).

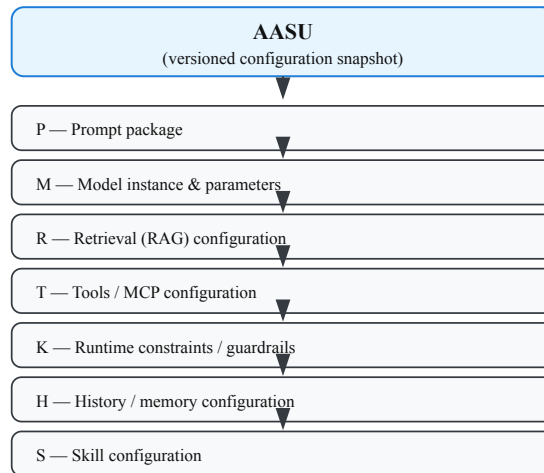


Figure 1. The Atomic AI Security Unit (AASU) as a configuration snapshot. Any change to P, M, R, T, K, H, S yields a distinct AASU that requires separate assurance.

Configuration Snapshot and Identity

Security posture is often *configuration-deterministic* even when generation outputs are probabilistic: small changes to prompts, tool schemas, retrieval thresholds, or guardrails can materially alter susceptibility to injection and

misuse. Therefore, security evaluation must bind to a concrete configuration snapshot.

We recommend representing each AASU as a canonical manifest (e.g., JSON) and computing an identifier via cryptographic hashing:

$$\text{AASU_ID} = H(\text{CanonicalSerialize}(P, M, R, T, K, H, S))$$

where H is a collision-resistant hash (e.g., SHA-256) applied to a deterministic serialization.

Illustrative Runtime Data Flow

Figure 2 depicts a typical request path through an AASU, highlighting the multiple decision points where configuration affects security posture.

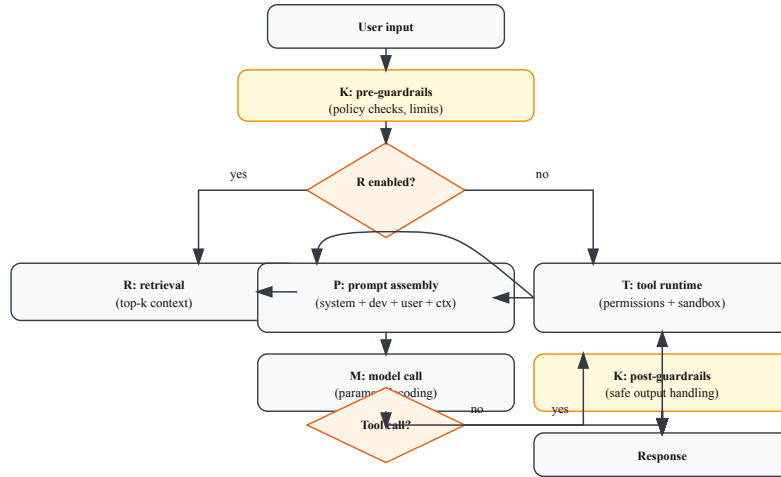


Figure 2. Illustrative runtime flow of an AASU. The attack surface spans retrieval, prompt assembly, model inference, tool execution, and guardrails.

Systems as Graphs of AASUs

Enterprise deployments often contain multiple AASUs linked by orchestration and routing. We model an application as a directed graph $G = (V, E)$ where each node $v \in V$ is an AASU configuration snapshot and each edge $e \in E$ represents a data flow relationship (messages, retrieved context, tool output reinjection, or cross-agent state sharing).

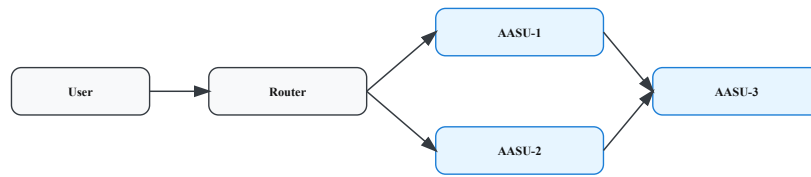


Figure 3. A directed graph of AASUs. Security analysis must consider both node properties (configuration) and edge properties (propagation and privilege pivots).

Edge-Scoped Risks

Even if individual nodes are robust in isolation, edge-level interactions can introduce vulnerabilities:

- **Injection propagation:** malicious instructions persist across nodes in sequential chains.
- **Privilege escalation chains:** lower-privilege nodes influence higher-privilege nodes via routing or shared memory.
- **Context contamination:** tool outputs and retrieved documents are reinjected without provenance or trust labeling.
- **Routing manipulation:** attackers steer queries toward weaker or more privileged AASUs.

Threat Modeling with AASU Components

We recommend threat models that explicitly enumerate assets, actors, trust boundaries, and data flows across P, M, R, T, K, H, S and across edges. This structure aligns naturally with security taxonomies such as OWASP Top 10 for LLM Applications (OWASP Foundation 2023) and MITRE ATLAS (MITRE 2020), while remaining configuration-aware.

Assets and Trust Boundaries

Common assets include sensitive data (PII, financial, proprietary), retrieval corpora and indices, tool capabilities (writes, refunds, exports), model credentials, and session memory/state. Trust boundaries frequently occur between: (i) user and application, (ii) application and model provider, (iii) orchestrator and tool runtime, and (iv) retrieval layer and data stores.

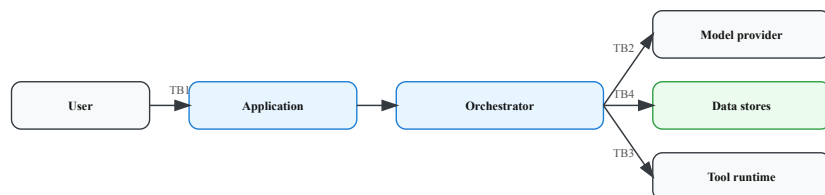


Figure 4. Typical trust boundaries in tool-using, retrieval-augmented LLM applications. AASU-based threat modeling makes these boundaries explicit and testable.

AASU Component Threat Surfaces

Table 1 summarizes representative risks and control themes by AASU component.

Table 1. Threat surfaces and control themes by AASU component.

Comp.	Representative risks	Control themes / evidence
<i>P</i>	Prompt injection; system prompt leakage; policy drift across templates and routing prompts	Prompt package versioning; instruction/context separation; policy tests bound to prompt hash
<i>M</i>	Model/provider drift; unsafe decoding settings; prompt-following behavior changes	Model version pinning; parameter constraints; repeated-trial testing and regression suites
<i>R</i>	Sensitive info disclosure; retrieval poisoning; instruction planting in corpora	Corpus allowlists; provenance labeling; retrieval filters; index versioning
<i>T</i>	Tool misuse; privilege escalation via tool chaining; schema abuse	Least privilege; allowlists; sandboxing; tool-call logging; schema versioning
<i>K</i>	Unbounded consumption (DoS); weak output handling; guardrail bypass	Rate limits/timeouts; safe output handling; guardrail enforcement telemetry

Representative Abuse Cases

We highlight abuse cases that are naturally expressed as AASU- and edge-scoped:

- **Prompt hierarchy override:** untrusted content attempts to supersede system/developer policies.
- **Tool misuse and parameter injection:** attacker induces unauthorized tool calls or malicious parameters.
- **Retrieval poisoning and instruction planting:** corpora contain adversarial instructions affecting downstream actions.
- **Cross-agent privilege pivot:** attacker leverages a weaker AASU to influence a more privileged AASU.

Mapping to OWASP and MITRE Taxonomies

OWASP Top 10 for LLM Applications provides a widely used risk taxonomy (OWASP Foundation 2023). Table 2 maps OWASP categories to AASU components and highlights the configuration-bound nature of the exposure.

Table 2. Illustrative mapping of OWASP Top 10 LLM risks to AASU components.

OWASP ID	Typical AASU components	Configuration-bound exposure examples
LLM01 Prompt Injection	P, R, T	Prompt templates; untrusted retrieval channels; tool-call policies
LLM02 Insecure Output Handling	K	Downstream actioning of text; missing output validation/sanitization
LLM06 Sensitive Info Disclosure	P, R, K	Over-broad retrieval; weak data minimization; unsafe logging
LLM07 Insecure Plugin Design	T	Over-privileged tools; ambiguous schemas; unsafe defaults
LLM08 Excessive Agency	T, K	Unbounded tool autonomy; missing approvals and rate limiting

MITRE ATLAS catalogs adversarial AI tactics and techniques (MITRE 2020). In AASU terms, many technique families can be localized to one or more of P, M, R, T, K, H, S and to graph edges (e.g., persistence via memory, lateral movement via cross-agent pivots), enabling consistent evidence collection across red team exercises and audits.

Three-Layer Validation Methodology

We propose a three-layer validation model intended to be implementable with existing test harnesses and compatible with enterprise governance.

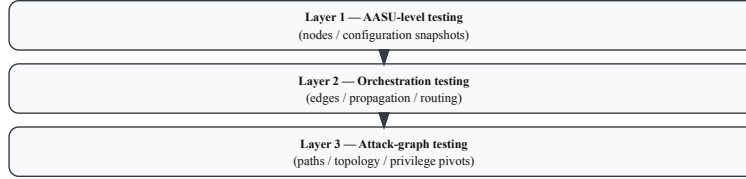


Figure 5. Three-layer validation. Unit robustness is necessary but insufficient; orchestration and topology must also be tested.

Layer 1: AASU-Level Testing

Layer 1 treats each node as a configuration-bound unit and tests: prompt injection resistance, tool misuse simulation, retrieval leakage validation, unsafe output handling, and robustness under obfuscation or multilingual bypass attempts. Tools such as garak (Derczynski et al. 2024) and PyRIT (Kumar et al. 2024) can be used as harnesses, but their results should be recorded against a specific AASU_ID.

Layer 2: Orchestration Testing

Layer 2 tests edge-level behavior: propagation of injections across nodes, retry-path exploitation, memory/state poisoning, and routing manipulation. This layer is essential for sequential and parallel agent fabrics (Figure 3).

Layer 3: Attack-Graph Testing

Layer 3 evaluates end-to-end attack paths, focusing on privilege escalation chains, cross-AASU exfiltration, and recursive tool invocation. Benchmarks such as InjecAgent (Liao et al. 2024) are relevant, but enterprise deployments require topology-specific scenarios.

Coverage Metrics

We recommend reporting coverage across both nodes and edges. For a deployed graph $G = (V, E)$, let $V_t \subseteq V$ be tested AASUs and $E_t \subseteq E$ tested edges. Define node and edge coverage:

$$C_V = \frac{|V_t|}{|V|}, \quad C_E = \frac{|E_t|}{|E|}$$

Path coverage can be tracked over selected critical paths (e.g., those that cross trust boundaries or increase tool privileges).

Governance and Auditability

The core governance requirement is that assurance outcomes are *version-bound* and *change-aware*. NIST AI RMF emphasizes risk management as an ongoing process rather than a one-time certification (National Institute of Standards and Technology 2023). AASU operationalizes this principle by binding tests and approvals to a configuration snapshot and requiring re-evaluation on change. At the artifact level, AASU manifests complement established documentation practices such as model cards and datasheets by extending them from component-level reporting to system-level, deployment-specific configuration snapshots (Mitchell et al. 2019; Gebru et al. 2018).

Lifecycle

We recommend the lifecycle in Figure 6: define/update configuration, generate a manifest, compute AASU_ID, run layered tests, perform risk review, deploy the certified snapshot, monitor and retain evidence, and trigger recertification on any change to P, M, R, T, K, H, S or key dependencies.

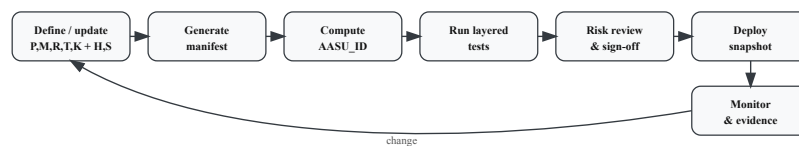


Figure 6. Governance lifecycle for AASU-based assurance. Any change to the configuration snapshot triggers re-evaluation and evidence retention supports audit readiness.

Example Manifest (Illustrative)

The manifest below shows a minimal sketch; in practice, this should also reference tool permission models, connector versions, and retrieval corpus identifiers.

Listing 1. Illustrative AASU manifest skeleton (JSON).

```
{
  "prompt_package": { "id": "P-2026-02-23-001", "hash": "...", },
  "model": { "provider": "...", "name": "...", "version": "...", "params": {
    "temperature": 0.2 } },
  "retrieval": { "enabled": true, "corpus_ids": ["KB-legal-v3"], "top_k": 8,
  "filters": ["..."] },
  "tools": { "enabled": true, "allowlist": ["search", "ticket_create"],
  "schemas_hash": "...",
  "constraints": { "max_tokens": 2048, "timeouts_ms": 10000,
  "rate_limit_rps": 2 }
}
```

Discussion and Limitations

Nondeterminism and reproducibility.

LLM outputs are stochastic; therefore, security evaluation should include repeated trials and adversarial search. However, AASU identity is still well-defined because it is derived from configuration, not from outputs.

Supply chain and drift.

Even with a stable configuration, upstream dependencies (model provider updates, embedding model changes, vector index rebuilds) can alter behavior. AASU governance should treat these as changes to *M* or *R* and trigger re-evaluation.

Empirical evaluation.

This paper focuses on formalization and governance. Future work should implement AASU manifests and coverage metrics in real enterprise pipelines and quantify improvements in defect localization and auditability.

Conclusion

We introduced the Atomic AI Security Unit (AASU) as a configuration-bound unit for security evaluation and governance of modern LLM applications that use tools and retrieval. By modeling deployed systems as graphs of AASUs and requiring layered validation across nodes, edges, and paths, AASU provides a practical foundation for repeatable red teaming, change-aware certification, and audit-ready evidence in enterprise GenAI systems.

References

- Anthropic. 2024. “Model Context Protocol (MCP).” Documentation.
<https://docs.anthropic.com/en/docs/agents-and-tools/mcp>.
- Arocas, Stéphane, Thomas I. L. Lembke, and Fredrik Heintz. 2024. “StruQ: Defending Against Prompt Injection with Structured Queries.” *CoRR* abs/2402.06363. <https://doi.org/10.48550/ARXIV.2402.06363>.
- Derczynski, Leon, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. 2024. “Garak: A Framework for Security Probing Large Language Models.” arXiv:2406.11036.
<https://doi.org/10.48550/arXiv.2406.11036>.
- European Union. 2024. “Regulation (EU) 2024/1689 of the European Parliament and of the Council (Artificial Intelligence Act).” Official Journal of the European Union. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>.
- Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2018. “Datasheets for Datasets.” *CoRR* abs/1803.09010.
<https://doi.org/10.48550/ARXIV.1803.09010>.

- Hartmann, Lukas P., and Iryna Gurevych. 2023. “BIPIA: A Benchmark for Evaluating the Robustness of LLMs Against Indirect Prompt Injection Attacks.” *CoRR* abs/2312.14197.
<https://doi.org/10.48550/ARXIV.2312.14197>.
- International Organization for Standardization. 2023. “ISO/IEC 42001: Artificial Intelligence — Management System.” ISO/IEC Standard.
<https://www.iso.org/standard/81230.html>.
- Jiang, Yue, Dengpan Ye, Shaonan Wang, Ziyang Pan, Yi Ren, Tianshuo Zhang, Tianyi Tang, et al. 2023. “CyberSecEval: A Comprehensive Security Benchmark for Large Language Models.” *CoRR* abs/2312.04724. <https://doi.org/10.48550/ARXIV.2312.04724>.
- Kumar, Ram Shankar Siva, David Valentin, Payton Boyd, Todd Maxwell, Wayne Blanchard, Aidan P. Naik, Daehoon Bae, et al. 2024. “PyRIT: A Tool for Finding Security Risks of Generative AI in the Wild.” *CoRR* abs/2410.02828. <https://doi.org/10.48550/ARXIV.2410.02828>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *CoRR* abs/2005.11401. <https://doi.org/10.48550/ARXIV.2005.11401>.
- Liao, Junjie, Bingjun Wang, Alan Y. Shih, Huan Sun, and Kamalika Chaudhuri. 2024. “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents.” *CoRR* abs/2403.02691. <https://doi.org/10.48550/ARXIV.2403.02691>.
- Liu, Yi, Hanwen Chen, Xiaojun Jia, Yang Yang, and Han Yu. 2023. “Prompt Injection Attacks Against LLM-Integrated Applications.” *CoRR* abs/2306.05499. <https://doi.org/10.48550/ARXIV.2306.05499>.
- Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. “Model Cards for Model Reporting.” *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
<https://doi.org/10.48550/ARXIV.1810.03993>.
- MITRE. 2020. “ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems.” MITRE. <https://atlas.mitre.org/>.
- Model Context Protocol. 2024. “Model Context Protocol Specification.” GitHub repository.
<https://github.com/modelcontextprotocol/specification>.
- National Institute of Standards and Technology. 2023. “AI Risk Management Framework 1.0.” NIST.
<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>.

- . 2024. “Artificial Intelligence Risk Management Framework: Generative AI Profile.” NIST. <https://doi.org/10.6028/NIST.AI.600-1>.
- OWASP Foundation. 2023. “OWASP Top 10 for LLM Applications.” OWASP. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. “Toolformer: Language Models Can Teach Themselves to Use Tools.” *CoRR* abs/2302.04761. <https://doi.org/10.48550/ARXIV.2302.04761>.
- Tramèr, Florian, Bo Li, Gautam Kamath, and Dan Boneh. 2025. “Defeating Prompt Injections by Design.” *CoRR* abs/2503.18813. <https://doi.org/10.48550/ARXIV.2503.18813>.
- Wu, Zhipeng, Xiaoyang Zhang, Zicheng Xu, Qingkai Zeng, Dan Shen, Siyuan Cheng, Guo Chen, Yufei Zhang, and Xusheng Xiao. 2024. “Aligning LLMs to Be Robust Against Prompt Injection.” *CoRR* abs/2410.05451. <https://doi.org/10.48550/ARXIV.2410.05451>.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. “ReAct: Synergizing Reasoning and Acting in Language Models.” *CoRR* abs/2210.03629. <https://doi.org/10.48550/ARXIV.2210.03629>.